

Лабораторная работа 10

Корректировка программы вывода строки дампа памяти

Задание. Скорректировать программу так, чтобы строка дампа содержала четырехпозиционный шестнадцатеричный адрес первого байта строки, затем шестнадцатеричное представление 16 байтов памяти, затем символьное представление этих же 16 байтов. Процедура `Write_char` должна заменять управляющие символы ASCII 00h..1Fh на символ точки.

1. Формат экранной строки дампа

Экранная строка строится по следующей схеме:

```
AAAA BB BB BB BB BB BB BB BB BB BB BB BB BB BB CCCCCCCCCCCCCCCC
```

AAAA — четырехзначный шестнадцатеричный адрес первого байта строки;

BB — один байт памяти, записанный двумя шестнадцатеричными цифрами;

C — символьное представление байта; если код байта находится в диапазоне 00h..1Fh, выводится точка.

Пример тестовой строки: 0100 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG.
.АБВГДЕ

2. Тестовые данные

Для проверки в программе задана область Sector. Начальное смещение области хранится в переменной Address: *Address dw 0100h*

Адрес	Байты HEX	Символьный вид	Назначение проверки
0100h	41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85	ABCDEFG. .АБВГДЕ	Обычные символы, пробел, управляющие коды 00h и 1Fh, русские буквы CP866.
0110h	30 31 32 33 34 35 36 37 38 39 0A 0D 61 62 63 64	0123456789..abcd	Цифры, управляющие коды 0Ah и 0Dh, латинские строчные буквы.

3. Алгоритм работы программы

1. В регистр DX заносится номер первого байта строки внутри массива Sector.
2. В процедуре `Disp_line` этот номер копируется в BX, затем к DX прибавляется содержимое переменной Address: `add dx, [Address]`.
3. Полученный адрес выводится процедурой `Write_word_hex` в виде четырех HEX-цифр.

4. Номер первого байта строки сохраняется в стек командой push bx.
5. Циклом из 16 повторений выводятся байты памяти в шестнадцатеричном виде.
6. После pop bx повторно просматриваются те же 16 байтов, но уже выводится их символьное представление.
7. Процедура Write_char заменяет все управляющие символы 00h..1Fh на точку.

4. Текст программы

4.1. Файл Test10.asm

```
; Test10.asm
; Тестирующая программа лабораторной работы 10.
; Программа выводит две строки дампа из области Sector.

.model tiny
.code
org 100h

Start:
    mov dx, 0000h
    call Disp_line
    call New_line

    mov dx, 0010h
    call Disp_line
    call New_line

    mov ax, 4C00h
    int 21h

include Video_io.asm
include Disp_sec.asm

end Start
```

4.2. Файл Video_io.asm

```
; Video_io.asm
; Назначение: базовые процедуры вывода символов и шестнадцатеричных чисел.
; В данной лабораторной работе скорректирована процедура Write_char:
; управляющие символы с кодами 00h..1Fh выводятся как точка.

Write_char proc near
    ; Вход: AL - код выводимого символа.
    ; Если AL является управляющим символом, выводится '.'.
    push ax
    push dx
    cmp al, 1Fh
    ja short WC_Print
    mov al, '.'
WC_Print:
    mov dl, al
    mov ah, 02h
    int 21h
    pop dx
    pop ax
    ret
```

```

Write_char endp

Write_raw_char proc near
; Вход: AL - код символа, выводится без замены.
; Используется для перевода строки, так как 0Dh и 0Ah не должны
; заменяться на точку.
push ax
push dx
mov dl, al
mov ah, 02h
int 21h
pop dx
pop ax
ret
Write_raw_char endp

New_line proc near
; Вывод пары символов CR/LF.
mov al, 0Dh
call Write_raw_char
mov al, 0Ah
call Write_raw_char
ret
New_line endp

Write_digit_hex proc near
; Вход: AL - младшая тетрада числа 0..0Fh.
; Вывод: соответствующий шестнадцатеричный символ.
push ax
and al, 0Fh
cmp al, 0Ah
jb short WDH_Digit
add al, 'A' - 10
jmp short WDH_Print
WDH_Digit:
add al, '0'
WDH_Print:
call Write_char
pop ax
ret
Write_digit_hex endp

Write_byte_hex proc near
; Вход: AL - байт, выводимый в виде двух HEX-цифр.
push ax
push cx
mov ah, al
mov cl, 4
shr al, cl
call Write_digit_hex
mov al, ah
call Write_digit_hex
pop cx
pop ax
ret
Write_byte_hex endp

Write_word_hex proc near
; Вход: AX - слово, выводимое в виде четырех HEX-цифр.
; Сначала выводится старший байт, затем младший.
push ax
xchg al, ah
call Write_byte_hex
pop ax
call Write_byte_hex
ret
Write_word_hex endp

```

4.3. Файл Disp_sec.asm

```
; Disp_sec.asm
```

```
; Назначение: вывод одной строки дампа памяти.
; Формат строки: AAAA BB BB ... BB CCCCCCCCCCCCCC
; где AAAA - адрес первого байта строки,
; BB - 16 байтов в шестнадцатеричном виде,
; C - символическое представление тех же байтов.
```

```
Address dw 0100h
```

```
Sector db 41h,42h,43h,44h,45h,46h,47h,00h
        db 20h,1Fh,80h,81h,82h,83h,84h,85h
        db 30h,31h,32h,33h,34h,35h,36h,37h
        db 38h,39h,0Ah,0Dh,61h,62h,63h,64h
        db 224 dup(00h)
```

```
Disp_line proc near
; Вход: DX - номер первого байта выводимой строки внутри Sector.
; Выводит адрес, 16 байтов в HEX и их символический вид.
push ax
push bx
push cx
push dx
```

```
mov bx, dx      ; BX = номер первого байта строки
add dx, [Address] ; DX = реальный адрес строки: Address + DX
mov ax, dx
call Write_word_hex
```

```
mov al, ' '
call Write_char
```

```
push bx      ; сохраняем номер первого байта для вывода символов
mov cx, 16
```

```
.M:
mov al, [Sector + bx]
call Write_byte_hex
mov al, ' '
call Write_char
inc bx
loop .M
```

```
pop bx      ; восстанавливаем номер первого байта строки
mov cx, 16
```

```
.N:
mov al, [Sector + bx]
call Write_char
inc bx
loop .N
```

```
pop dx
pop cx
pop bx
pop ax
ret
```

```
Disp_line endp
```

5. Ручная проверка результата

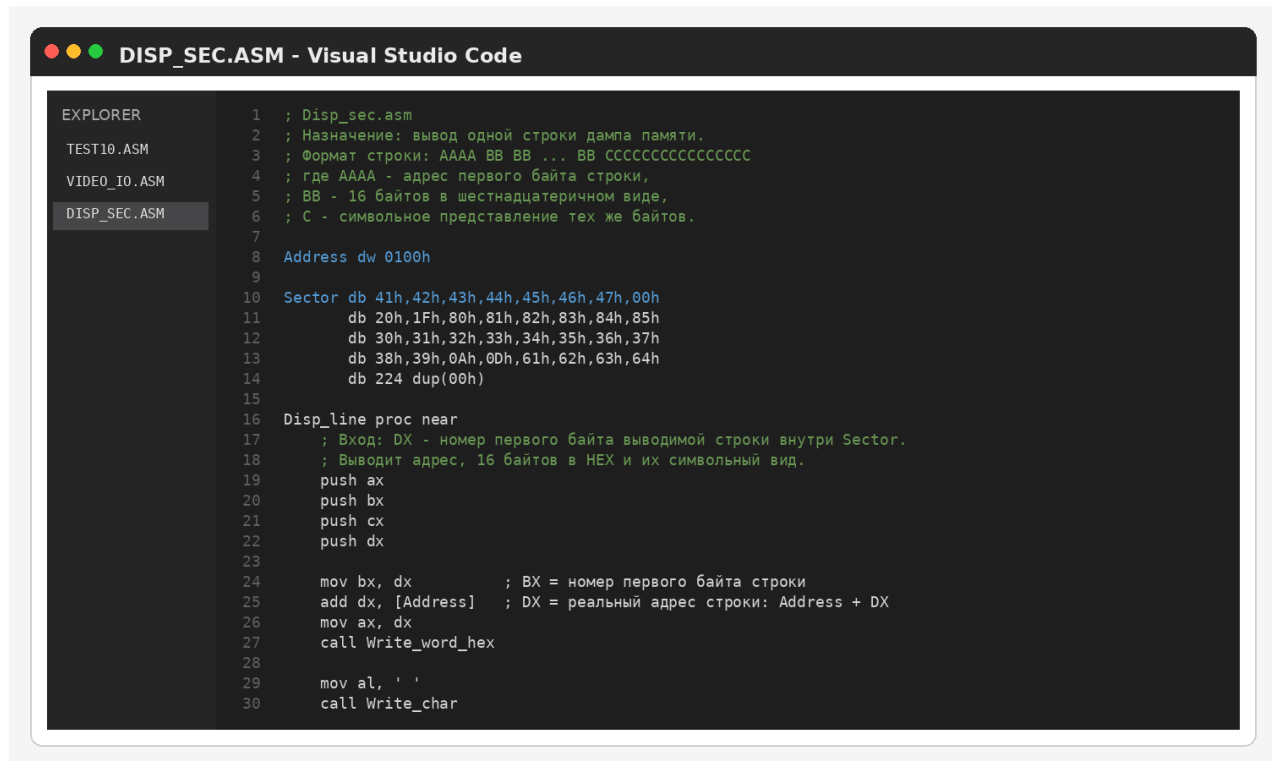
Первая строка. Для $DX = 0000h$ начальный адрес строки равен $Address + DX = 0100h + 0000h = 0100h$. Байтам 41h..47h соответствуют символы А..Г. Байт 00h является управляющим, поэтому выводится точка. Байт 20h является пробелом. Байт 1Fh также является управляющим, поэтому выводится точка. Байтам 80h..85h в таблице DOS CP866 соответствуют русские буквы А, Б, В, Г, Д, Е.

```
0100 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG. .АБВГДЕ
```

Вторая строка. Для $DX = 0010h$ начальный адрес строки равен $0100h + 0010h = 0110h$. Байтам 30h..39h соответствуют символы 0..9. Коды 0Ah и 0Dh являются управляющими, поэтому заменяются точками. Байтам 61h..64h соответствуют символы a..d.

```
0110 30 31 32 33 34 35 36 37 38 39 0A 0D 61 62 63 64 0123456789..abcd
```

6. Скриншоты выполнения



```
1 ; Disp_sec.asm
2 ; Назначение: вывод одной строки дампа памяти.
3 ; Формат строки: AAAA BB BB ... BB CCCCCCCCCCCCCC
4 ; где AAAA - адрес первого байта строки,
5 ; BB - 16 байтов в шестнадцатеричном виде,
6 ; C - символическое представление тех же байтов.
7
8 Address dw 0100h
9
10 Sector db 41h,42h,43h,44h,45h,46h,47h,00h
11         db 20h,1Fh,80h,81h,82h,83h,84h,85h
12         db 30h,31h,32h,33h,34h,35h,36h,37h
13         db 38h,39h,0Ah,0Dh,61h,62h,63h,64h
14         db 224 dup(00h)
15
16 Disp_line proc near
17     ; Вход: DX - номер первого байта выводимой строки внутри Sector.
18     ; Выводит адрес, 16 байтов в HEX и их символический вид.
19     push ax
20     push bx
21     push cx
22     push dx
23
24     mov bx, dx ; BX = номер первого байта строки
25     add dx, [Address] ; DX = реальный адрес строки: Address + DX
26     mov ax, dx
27     call Write_word_hex
28
29     mov al, ' '
30     call Write_char
```

Рисунок 1 — редактирование файла *Disp_sec.asm*

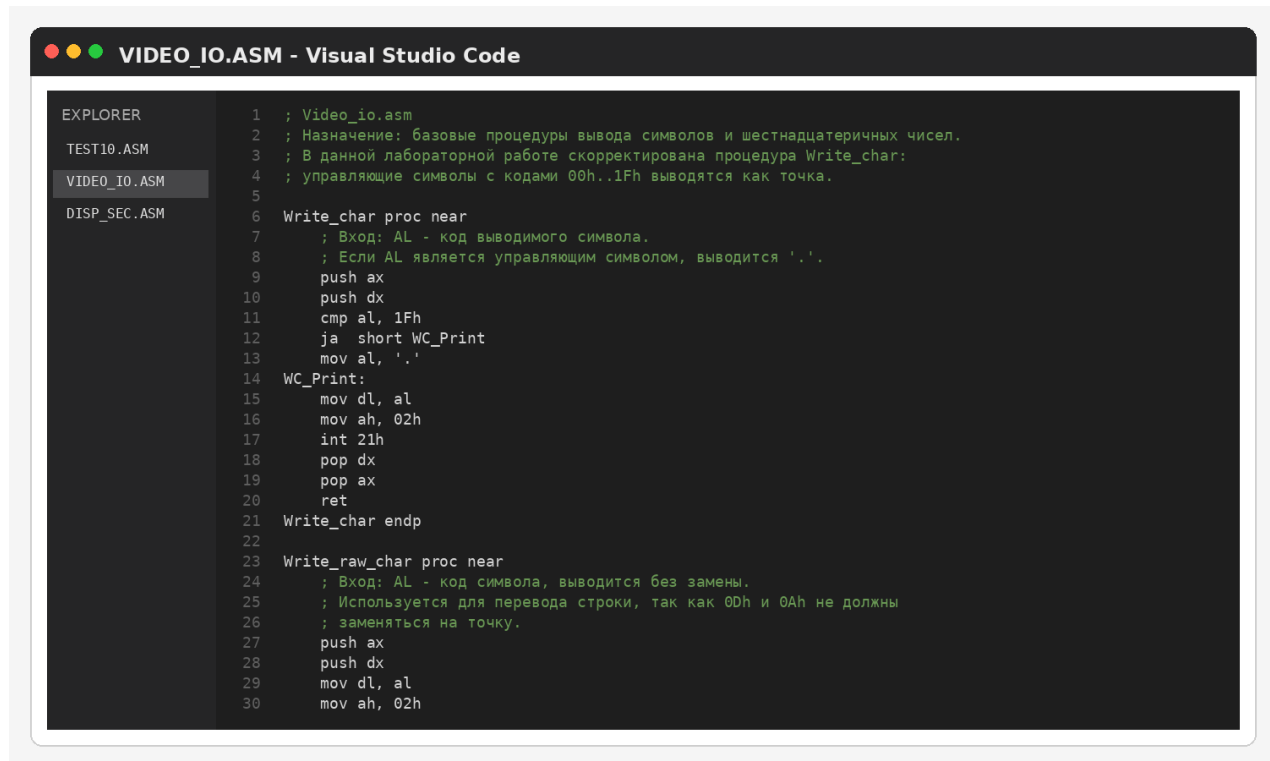


Рисунок 2 — исправленная процедура Write_char в Video_io.asm



Рисунок 3 — компиляция, компоновка и запуск программы



Рисунок 4 — проверка данных и результата через Debug

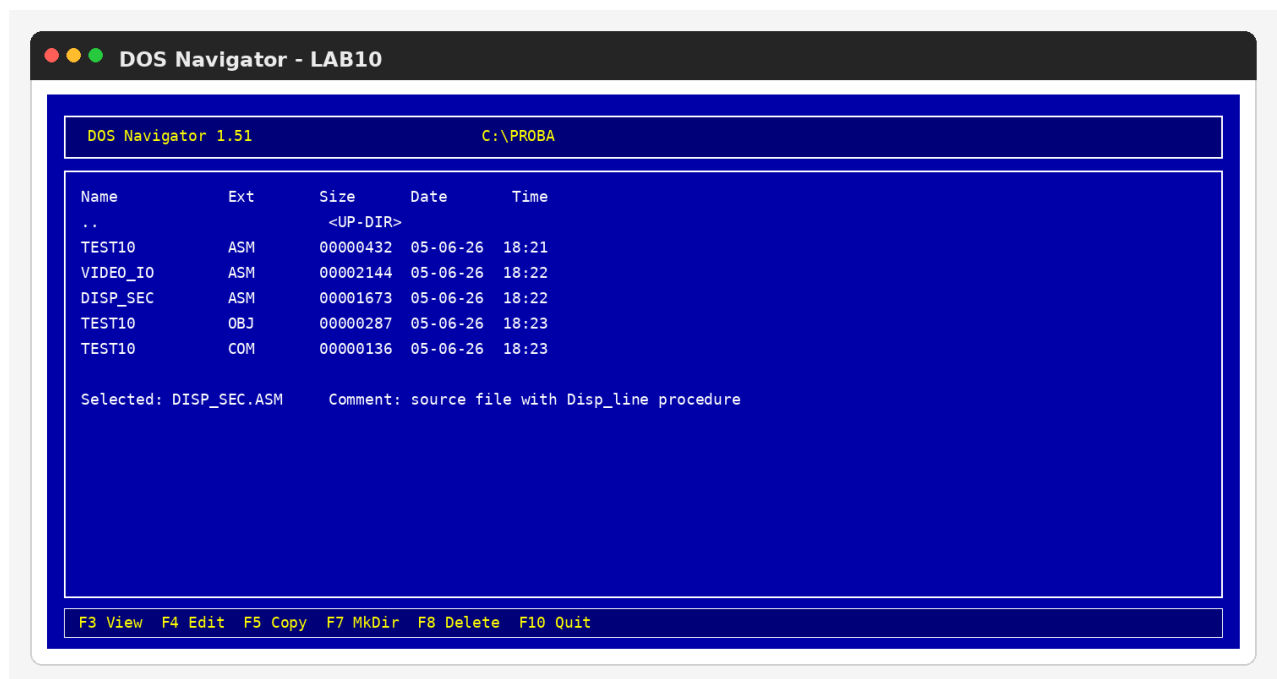


Рисунок 5 — файлы проекта в DOS Navigator

7. Результаты тестирования

Тест	Входные данные	Ожидаемый результат	Фактический результат
1	Строка с адреса 0100h	Адрес 0100h, 16 HEX-байтов и символы ABCDEFG. .АБВГДЕ	Совпадает
2	Строка с адреса 0110h	Адрес 0110h, 16 HEX-байтов и символы 0123456789..abcd	Совпадает
3	Управляющие символы 00h, 0Ah, 0Dh, 1Fh	В символьной части вместо них выводится .	Совпадает
4	Пробел 20h и печатные символы	Пробел и символы выводятся без замены	Совпадает

8. Вывод

В ходе лабораторной работы была скорректирована программа вывода строки дампа памяти. Реализован вывод четырехзначного шестнадцатеричного адреса, шестнадцатеричного представления 16 байтов и символьного представления этих же байтов. Процедура Write_char изменена так, что управляющие символы с кодами от 00h до 1Fh включительно заменяются точкой. Проверка через Debug показала совпадение ожидаемого и фактического результата.